课程一：Python编程基础

源自：南京邮电大学计算机学院郭贤杰老师

引言

本课程是"科研竞赛入门培训"系列的**第一门课**，面向零基础学员。之所以把Python基础放在最前面，是因为后续所有的AI模型嵌入、机器学习算法设计、数据预处理、算法实验运行与结果分析，都建立在"能自己写出一段可运行、可调试、可复用的代码"这一能力之上。很多同学在接触AI算法与机器学习时会发现：**调包容易，写代码难**——一旦离开现成的库函数，就不知道该如何组织一段程序、如何设计一个类、如何处理一个文件。本课程的目标正是先补上这块短板：通过系统学习Python的语法体系、数据结构、面向对象机制与常用标准库，让你从"会调用"进阶到"会实现"。

本课程的视频学习内容主要来自B站【Python 基础教程】（莫烦Python，共36节，总时长约4小时）。配套作业**全部要求手写实现核心逻辑，禁止直接调用现成的高级库函数或利用大模型完成作业（要对自己负责任）**，以此真正锻炼编程思维与代码能力。打好这一步的基础后，你在课程二中使用sklearn完成回归与分类实验时，才不会停留在"复制粘贴、不知其所以然"的层面。

本次课程的主要内容包括：

1. 开发环境搭建与工具准备
2. Python基础语法（第1~4节）
3. 程序控制流程与函数（第5~13、27节）
4. Python核心数据结构（第20~24、30、35节）
5. 模块化、文件读写与面向对象（第14~19、25、26、28、34节）
6. Python进阶特性与常用工具（第29、31~33、36节）
7. 完成课程作业（5个手写代码项目）

内容一：开发环境搭建与工具准备

- Python开发环境配置：Python解释器安装 + VS Code开发工具安装
- VS Code的基本使用：教程视频
- 视频课程中的环境搭建部分：第1节 安装（07:25）

- **学习要点**：确认解释器版本（建议Python 3.8及以上）并配置好环境变量，在终端中能够正确执行 `python --version` 与 `pip --version`；掌握VS Code中Python插件的安装、解释器的选择、代码运行与断点调试的基本操作。调试能力（打断点、单步执行、查看变量）比记住语法更重要，请务必在本阶段练熟

内容二：Python基础语法

- 第一行代码与输出控制：第2节 print()（08:46）
- 数学运算与运算符：第3节 数学（03:40）
- 变量的定义与命名规范：第4节 自变量 variable（06:13）
- 补充参考：基础语法与数据类型
- **学习要点**：理解Python作为解释型语言、动态类型语言的含义；掌握 `print()` 的多参数输出与三种格式化写法（ `%` 格式化、 `str.format()` 、f-string）之间的区别与适用场景；理解整除 `//` 、取余 `%` 、幂运算 `**` 的行为，尤其注意浮点数精度问题（可思考为什么 `0.1 + 0.2 != 0.3` ，细节可咨询Claude（课题组内部会用使用链接）、Gemini、DeepSeek、ChatGPT 等大语言模型）

内容三：程序控制流程与函数

循环与分支

- 第5节 while循环（06:04）、第6节 for循环（10:48）
- 第7节 if 条件（06:32）、第8节 if else 条件（03:05）、第9节 if elif else（06:08）
- 第27节 continue & break（05:51）

函数与作用域

- 第10节 def 函数（06:30）、第11节 函数 参数（04:23）、第12节 函数默认参数（09:47）
- 第13节 全局 & 局部变量, global & local（09:47）
- 补充参考：程序控制流程、函数与模块化编程
- **学习要点**：理解 `range()` 的三种用法与"左闭右开"区间约定；分清 `break` （跳出整个循环）与 `continue` （跳过本次迭代）；掌握位置参数、关键字参数、默认参数的传参顺序规则；**特别注意**默认参数为可变对象（如列表）时的陷阱，理解为什么 `def f(x, lst=[])` 是危险写法；理解LEGB作用域查找规则，以及 `global` 关键字的适用场景与滥用风险

内容四：Python核心数据结构

- 交互式输入：第20节 input 输入（07:00）
- 序列类型：第21节 元组 列表（08:25）、第22节 列表 list（11:57）、第23节 多维列表（06:13）
- 映射类型：第24节 字典 dictionary（07:49）
- 集合类型：第35节 set 找不同（12:29）
- 对象复制机制：第30节 浅复制 & 深复制, copy & deepcopy（08:38）
- 补充参考：核心数据结构
- **学习要点**：理解可变对象（list、dict、set）与不可变对象（int、str、tuple）的本质区别；掌握列表的切片、增删改查与列表推导式；理解"字典的键必须可哈希"这一约束的来由；掌握集合的交集 `&`、并集 `|`、差集 `-`、对称差集 `^`；**重点理解**赋值（`b = a`）、浅拷贝（`copy.copy`）、深拷贝（`copy.deepcopy`）三者在多维列表上的行为差异——这是作业四的核心考点，也是日后处理数据集时最容易踩坑的地方

内容五：模块化、文件读写与面向对象

模块与包管理

- 第14节 模块安装（07:35）、第25节 import 载入模块（06:08）、第26节 自己的模块（06:21）

文件读写与数据持久化

- 第15节 读写文件1（06:28）、第16节 读写文件2（03:23）、第17节 文件读写3（06:05）
- 第34节 pickle 存放数据（06:01）

面向对象与异常处理

- 第18节 class 类（11:12）、第19节 类 init 功能（10:12）
- 第28节 错误处理 try（06:58）
- **学习要点**：掌握 `pip install` 与国内镜像源配置（这直接决定了课程二中能否顺利安装sklearn等第三方包）；理解 `import x`、`from x import y`、`from x import *` 的区别，以及 `if __name__ == '__main__'` 的作用；掌握 `r` / `w` / `a` / `r+` 四种文件打开模式的差异，养成使用 `with open(...)` 上下文管理器的习惯；理解 `self` 的含义、实例属性与类属性的区别；掌握 `try/except/else/finally` 结构，学会捕获**具体异常类型**而非使用裸 `except`

内容六：Python进阶特性与常用工具

- 函数式编程工具：第29节 zip lambda map（07:47）
- 文本模式匹配：第36节 RegEx 正则表达式

- 并发与界面（了解即可）：第31节 多线程 threading（02:17）、第32节 multiprocessing 介绍（01:44）、第33节 tkinter 窗口（01:56）
- 综合练习：Python编程练习题
- **学习要点**：掌握 `lambda` 匿名函数的适用边界（仅限简单表达式）与 `map / filter / zip` 的惰性求值特性；掌握正则表达式的元字符、字符类、量词、分组，并分清 `re.match / re.search / re.findall / re.sub` 的区别；了解GIL（全局解释器锁）的存在，理解为什么Python多线程适合IO密集型任务、而多进程适合CPU密集型任务（细节可咨询Claude（课题组内部会用使用链接）、Gemini、DeepSeek、ChatGPT 等大语言模型）

内容七：完成课程作业

总体要求：以下5个作业**全部要求手写实现核心逻辑**。除各题明确允许的标准库外，禁止使用numpy、pandas、collections.Counter、内置排序函数、math库中与题目核心逻辑重复的函数等任何"一行搞定"的现成方案。每个作业单独建立一个文件夹（ `homework1` ~ `homework5` ），代码需包含必要注释，并附一份 `README.md` 说明运行方式与设计思路。

作业一：手写基础算法工具箱（对应第1~13、27节）

- ☐ 新建模块 `mymath.py` ，**不导入任何第三方库，也不使用 `math` 库**，手写实现以下函数：
  1. `is_prime(n)` ：判断n是否为素数，要求将循环上界优化到 `sqrt(n)` （平方根请用循环或牛顿迭代自行求出，不许调用 `math.sqrt` ）
  2. `gcd(a, b)` 与 `lcm(a, b)` ：用辗转相除法求最大公约数，并由此推出最小公倍数
  3. `my_sqrt(x, eps=1e-8)` ：用**牛顿迭代法**求平方根， `eps` 为默认参数，用于控制精度
  4. `fib(n, method='loop')` ：求斐波那契数列第n项，通过默认参数 `method` 支持 `'loop'` （循环）与 `'recursion'` （递归）两种实现方式
  5. `prime_list(start, end)` ：返回区间内所有素数组成的列表，要求内部调用 `is_prime`
- ☐ 新建 `main1.py` 导入上述模块，完成：
  1. 打印规范对齐的九九乘法表（下三角形式）
  2. 打印一个用户指定行数的**空心菱形**图案（需综合运用嵌套循环与if判断）
  3. 用 `while True` + `break / continue` 实现一个循环菜单：用户输入数字选择功能，输入 `q` 退出，输入非法内容时给出提示并 `continue`
- ☐ **思考题（写入README）**：递归版 `fib(35)` 与循环版 `fib(35)` 耗时相差多少？为什么？

作业二：手写文本词频统计器（对应第15~17、21~24、35节）

- ☐ 自行准备一个不少于2000词的英文文本文件 `corpus.txt` （可从古登堡计划等公开语料下载），编写 `wordcount.py` 完成：
  1. 读取文件（必须使用 `with open` ），统一转为小写，**手写函数**去除标点符号并按空白切分成单词列表（不许用正则，本题专门练习纯字符串操作）
  2. **手写实现**词频统计：用字典累加，禁止使用 `collections.Counter`
  3. **手写实现排序算法**（选择排序、插入排序或快速排序任选其一），按词频降序输出Top 20，禁止使用内置 `sorted()` 和 `list.sort()`
  4. 读取给定的停用词表 `stopwords.txt` ，用 `set` 做差集运算过滤停用词，对比过滤前后Top 20的变化
  5. 将统计结果以 `单词,词频` 的CSV格式写入 `result.csv` （手写字符串拼接，不用csv库）
- ☐ **进阶要求**：统计文本中出现的所有**二元词组**（bigram，即相邻两个单词的组合），输出出现次数最多的10个
- ☐ **思考题（写入README）**：为什么用字典统计词频比用两层循环遍历列表快得多？请从时间复杂度角度分析

作业三：面向对象——学生成绩管理系统（对应第18~20、25、26、28、34节）

- ☐ 项目至少拆分为三个文件，体现模块化设计：

文件名	职责
`models.py`	定义 `Student` 类与 `ClassRoom` 类
`storage.py`	负责pickle存档与读档
`main.py`	命令行交互主程序

- ☐ `Student` 类要求：用 `__init__` 初始化学号、姓名、各科成绩字典；提供 `average()` 、 `total()` 、 `highest_subject()` 方法；实现 `__str__` 方法，使 `print(student)` 能输出可读信息
- ☐ `ClassRoom` 类要求：内部用列表或字典管理多个 `Student` 对象；提供增（ `add` ）、删（ `remove` ）、改（ `update` ）、查（ `find_by_id` 、 `find_by_name` ）方法；提供 `rank()` 方法按总分排序输出排名（排序算法仍需手写）；提供 `subject_stats(subject)` 返回某科目的平均分、最高分、最低分与及格率

- ▢ 主程序要求：用 `input()` 实现循环菜单交互；**所有用户输入必须用 `try/except` 保护**，至少正确处理 `ValueError`（输入非数字）、`KeyError`（学号不存在）、`FileNotFoundError`（首次运行无存档）三类异常；程序退出时用 `pickle` 自动保存数据，启动时自动载入
- ▢ **思考题（写入README）：** `pickle` 存储对象与存储纯文本CSV相比，各有什么优缺点？pickle文件存在什么安全隐患？

作业四：手写矩阵运算库（对应第22、23、30节）

- ▢ 编写 `matrix.py`，用**嵌套列表**表示矩阵，全程禁止使用numpy，手写实现：
  1. `zeros(m, n)`、`eye(n)`：生成零矩阵与单位矩阵（注意避开 `[[0]*n]*m` 这一经典陷阱，请在README中解释该写法为何错误）
  2. `add(A, B)`、`sub(A, B)`：矩阵加减，维度不匹配时 `raise ValueError` 并给出清晰的错误信息
  3. `matmul(A, B)`：矩阵乘法，用三层循环实现
  4. `transpose(A)`：转置（请分别用嵌套循环与 `zip(*A)` 两种方式实现并对比）
  5. `det(A)`：**递归**实现行列式计算（按第一行代数余子式展开）
  6. `inverse(A)`：用高斯-约当消元法求逆矩阵，遇到奇异矩阵时抛出异常
- ▢ **深浅拷贝专项验证：**编写 `copy_test.py`，构造一个3×3嵌套列表，分别用 `B = A`、`B = A.copy()`、`B = copy.deepcopy(A)` 三种方式复制后修改 `B[0][0]`，打印 `A` 的变化并解释原因。说明为什么在 `inverse()` 的消元过程中**必须**使用 `deepcopy`
- ▢ **验证要求：**随机构造若干矩阵，验证 `matmul(A, inverse(A))` 的结果是否接近单位矩阵（自行设定浮点容差）
- ▢ **衔接提示：**本作业手写的矩阵运算，正是课程二中线性回归等算法的底层计算基础。完成后，你再看sklearn的 `fit` 与 `predict` 就不再是黑箱

作业五：综合项目——服务器日志分析工具（对应第28、29、31~36节）

- ▢ 自行构造或下载一份Web服务器访问日志 `access.log` （不少于500行，格式参考：`192.168.1.10 - - [07/Apr/2025:23:19:48 +0800] "GET /index.html HTTP/1.1" 200 5124` ），编写 `log_analyzer.py` 完成：
  1. 用**正则表达式**（ `re` 模块）设计带分组的模式，一次性提取每行的IP、时间、请求方法、URL路径、状态码与响应字节数；无法匹配的脏数据行用 `try/except` 跳过并计数
  2. 用 `map` 与 `lambda` 将提取出的字节数字符串批量转为整数，并将状态码归类为 `2xx/3xx/4xx/5xx`
  3. 用 `zip` 将IP列表与状态码列表配对，统计**每个IP的错误请求（4xx/5xx）次数**，输出Top 5
  4. 用 `set` 运算找出：只在上午出现过的IP、只在下午出现过的IP、全天都在访问的IP
  5. 用 `re.sub` 对日志做脱敏处理（将IP最后一段替换为 `***` ），输出 `access_masked.log`
  6. 生成分析报告写入 `report.txt` ，包含总请求数、独立IP数、各状态码占比、流量总计、访问量Top 10的URL
- ▢ **选做加分项（任选其一）：**
  - 用 `threading` 同时分析多个日志文件并合并结果，与串行版本对比耗时，在README中结合GIL解释观察到的现象
  - 用 `tkinter` 做一个简易图形界面：选择日志文件 → 点击分析 → 文本框展示报告
- ▢ **思考题（写入README）：**你写的正则表达式在遇到URL中包含引号或空格的日志行时会失效吗？请构造一个反例并给出改进方案

作业清单一览

作业编号	主题	覆盖课节	核心手写要求
作业一	基础算法工具箱	1~13、27	牛顿迭代、辗转相除、递归与循环
作业二	文本词频统计器	15~17、21~24、35	字典统计、手写排序、集合过滤
作业三	成绩管理系统	18~20、25、26、28、34	类设计、异常处理、pickle持久化
作业四	矩阵运算库	22、23、30	多维列表、递归行列式、深浅拷贝
作业五	日志分析工具	28、29、31~36	正则提取、map/lambda/zip、

课程作业要求：

> 请在**课程一截止日期前**（11月13日）将5个作业的完整Python项目代码（每个作业一个独立文件夹，内含源码、测试数据、README.md）连同一份汇总说明文档一起打包压缩，发送至邮箱：xianjieguo@njupt.edu.cn，压缩包命名方式为：**"科研竞赛入门培训+课程一+姓名"**。
> **提示**：遇到语法或报错问题，可先自行查阅Python官方文档，再咨询Claude（课题组内部会用使用链接）、Gemini、DeepSeek、ChatGPT等大语言模型。但请注意：本课程的目的是训练你的编程能力，直接让大模型生成完整作业代码将使本课程失去意义。